VELLORE INSTITUTE OF TECHNOLOGY
VELLORE CAMPUS

MPMC LAB

Task 5 — Experiment Report

Registration Number	:	24BCE0696
Name	:	Patel Purav Nitinkumar
Experiment Number	:	5
Date	:	01 April 2026
Status	:	HOSTELLER

Question 1

Write an Assembly Language Program (ALP) to execute basic arithmetic operations (Addition, Subtraction, Multiplication, and Division) on an ARM7 processor and verify the results using Keil µVision simulator.

AIM

To write, compile, and simulate an Assembly Language Program (ALP) for performing basic arithmetic operations — Addition, Subtraction, Multiplication, and Division — on an ARM7 processor using Keil µVision software, and to verify the outputs using the built-in simulator.

APPARATUS REQUIRED

- Keil µVision IDE (with ARM7 device support)
- Personal Computer / Laptop
- ARM7 Simulator (integrated in Keil µVision)

ALGORITHM

a) Addition

1. Start
2. Load the first operand (0x03) into register R0
3. Load the second operand (0x02) into register R1
4. Add R0 and R1; store the result in R2
5. Stop

b) Subtraction

1. Start
2. Load the first operand (0x06) into register R0
3. Load the second operand (0x02) into register R1
4. Subtract R1 from R0; store the result in R2
5. Stop

c) Multiplication

1. Start
2. Load the first operand (0x03) into register R0
3. Load the second operand (0x02) into register R1

4. Multiply R0 and R1 using MUL; store the result in R3

5. Stop

d) Division

1. Start

2. Load dividend (0x05) into R1

3. Load divisor (0x02) into R2

4. Initialize quotient (R3) = 0

5. Compare R1 and R2

6. If R1 < R2 → Stop (division complete)

7. Else subtract R2 from R1

8. Increment quotient (R3)

9. Repeat from step 5

10. Stop

CODE

---

a) Addition

```
    AREA addition, CODE, READONLY
    MOV R0, #0x03
    MOV R1, #0x02
    ADD R2, R1, R0
    L B L
    END
```

b) Subtraction

```
    AREA subtraction, CODE, READONLY
    MOV R0, #0x06
    MOV R1, #0x02
    SUB R2, R0, R1
    L B L
    END
```

c) Multiplication

```
    AREA multiplication, CODE, READONLY
    MOV R0, #0x03
    MOV R1, #0x02
    MUL R3, R1, R0
    L B L
```

```
    END
```

d) Division

```
    AREA division, CODE, READONLY
    MOV R1, #0x05
    MOV R2, #0x02
L1  CMP R1, R2
    BCC L
    SUB R1, R2
    ADD R3, R3, #0x01
    B L1
L   B L
    END
```

RESULT

The Assembly Language Programs for Addition, Subtraction, Multiplication, and Division were successfully written, compiled, and simulated on the ARM7 processor using Keil µVision. The computed results were verified to be correct using the simulator's register watch window.

Question 2

Write an Assembly Language Program to generate a Sawtooth waveform using DAC interfaced with an ARM7 processor and verify on a processor board.

AIM

To write and execute an Assembly Language Program for generating a Sawtooth waveform by interfacing a DAC (Digital-to-Analog Converter) with an ARM7 processor, and to observe the waveform output on the processor board.

APPARATUS REQUIRED

• Keil µVision IDE

• ARM7 Processor Board (LPC2148 or equivalent)

• DAC Module (interfaced with ARM7)

• Oscilloscope (for waveform observation)

• Personal Computer / Laptop

ALGORITHM

1. Start
2. Initialize the DAC and configure PINSEL register
3. Set the output value A = 0
4. Output value A to the DAC
5. Increment A by 1
6. If A < MAX value → go to step 4
7. Else reset A = 0
8. Repeat from step 4 continuously

CODE

```
    AREA DAC, CODE, READONLY
PINSEL1  EQU 0xE002C006
DACR     EQU 0xE006C000
         LDR R3 = 0x00000 3FF
         LDR R1 = PINSEL1
         LDR R2 = 0x0C000000
         STR R2, (R1)
         LDR R0 = DACR
```

```
        LDR R2 = 0x0
DtoA
        LDL R1, R2, R6
        STR R1, (R2)
        ADD R2, R2, R1
        CMP R2, R3
        BLT DtoA
        LDR R2 = 0x0
        B   DtoA
        END
```

RESULT

The Assembly Language Program for generating a Sawtooth waveform was successfully written and executed on the ARM7 processor. The sawtooth pattern — linearly increasing from 0 to maximum and then resetting — was observed and verified on the processor board.

Question 3

Write an Assembly Language Program to generate a Triangular waveform using DAC interfaced with an ARM7 processor and verify on a processor board.

AIM

To write and execute an Assembly Language Program for generating a Triangular waveform by interfacing a DAC with an ARM7 processor, and to observe the output on the processor board.

APPARATUS REQUIRED

• Keil µVision IDE

• ARM7 Processor Board (LPC2148 or equivalent)

• DAC Module (interfaced with ARM7)

• Oscilloscope (for waveform observation)

• Personal Computer / Laptop

ALGORITHM

1. Start
2. Initialize DAC and configure PINSEL register
3. Set A = 0
4. (Rising Phase) Output A to DAC, increment A
5. If A < MAX → repeat step 4
6. (Falling Phase) When MAX is reached, decrement A, output A to DAC
7. If A > 0 → repeat step 6
8. When A = 0, repeat from step 4

CODE

```
    AREA trianglewave, CODE, READONLY
PINSEL1  EQU 0xE002C006
DACR     EQU 0xE006C000
         LDR R3 = 0x000003FF
         LDR R1 = PINSEL1
         LDR R0 = DACR
         LDR R2 = 0
         STR R1, (R0)
         LDR R2 = 0x0C000000
```

```
        STR R2, (R0)
RampAp
        LDL R1, R2, R6
        STR R1, (R2)
        ADD R1, R3, #1
        CMP R2, R3
        BLT RampAp
RampDn
        LDL R1, R2, R6
        STR R1, (R2)
        SUBS R2, R2, #1
        BNE RampDn
        B   RampAp
        END
```

RESULT

The Assembly Language Program for generating a Triangular waveform was successfully written and executed on the ARM7 processor. The waveform — gradually rising to maximum and symmetrically falling back to zero — was observed and verified on the processor board.

Question 4

Write an Assembly Language Program to generate a Square waveform using DAC interfaced with an ARM7 processor and verify on a processor board.

AIM

To write and execute an Assembly Language Program for generating a Square waveform by interfacing a DAC with an ARM7 processor, and to observe the output on the processor board.

APPARATUS REQUIRED

- Keil µVision IDE
- ARM7 Processor Board (LPC2148 or equivalent)
- DAC Module (interfaced with ARM7)
- Oscilloscope (for waveform observation)
- Personal Computer / Laptop

ALGORITHM

1. Start
2. Initialize DAC, configure PINSEL and port registers
3. Output HIGH value (0xFFFFFFFF) to DAC
4. Wait for delay
5. Output LOW value (0x00) to DAC
6. Wait for delay
7. Repeat from step 3 continuously

CODE

```
    AREA mname, CODE, READONLY
IODIR    EQU 0x3F028004
IOSET    EQU 0x3F028014
IOCLR    EQU 0x3F02801C
IOPIN    EQU 0x3F028010
START
         LDR R0 = IODIR
         LDR R1 = 0xC2000000
         STR R1, (R0)
         LDR R3 = IOSET
```

```
        STR R1, (R3)
        BL  DELAY
        LDR R2 = IOCLR
        STR R1, (R2)
        BL  DELAY
        B   START
DELAY
        LDR R4 = 0XC0F0000F
L       SUBS R4, R4, #0x01
        BNE  L
        BX   LR
        END
```

RESULT

The Assembly Language Program for generating a Square waveform was successfully written and executed on the ARM7 processor. The square wave output — alternating between HIGH and LOW states with a human-perceivable delay — was observed and verified on the processor board.

Question 5

Write an Assembly Language Program to interface a buzzer with an ARM7 processor and make the buzzer turn ON and OFF with a human-perceivable delay.

AIM

To write and execute an Assembly Language Program to interface a buzzer with the ARM7 processor such that the buzzer toggles ON and OFF with a human-perceivable delay, using GPIO port control registers.

APPARATUS REQUIRED

• Keil µVision IDE

• ARM7 Processor Board (LPC2148 or equivalent)

• Buzzer module (interfaced via GPIO port)

• Personal Computer / Laptop

• Connecting wires and power supply

ALGORITHM

1. Start
2. Configure the GPIO port as output using IODIR register
3. Turn buzzer ON by writing to IOSET register
4. Call DELAY subroutine
5. Turn buzzer OFF by writing to IOCLR register
6. Call DELAY subroutine
7. Repeat from step 3 continuously

CODE

```
CODE    AREA buzzer, CODE, READONLY
IODIR   EQU  0xE0028014
IOSET   EQU  0xE0028014
IOCLR   EQU  0xE002801C
IOPIN   EQU  0xE0028010
START
        LPR  R0 = IODIR
        LDR  R1 = 0xC2000000
        STR  R1, (R0)
```

```
        LDR  R3 = IOSET
        STR  R1, (R3)
        BL   DELAY
        LDR  R2 = IOCLR
        STR  R1, (R2)
        BL   DELAY
        B    START
DELAY
        LDR  R4 = 0XCOF0000F
L       SUBS R4, R4, #0x01
        BNE  L
        BX   LR
        END
```

RESULT

The Assembly Language Program for interfacing a buzzer with the ARM7 processor was successfully written, compiled, and executed. The buzzer toggled ON and OFF with a clear, human-perceivable delay, confirming correct GPIO control and interrupt-free delay loop operation.